FACULTÉ DES SCIENCES DHAR EL MAHRAZ
UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH

UNIVERSITÉ SORBONNE PARIS NORD

Master Web Intelligence
et Science de données

Projet de fin de module : Programmation avancée et technologies JEE

Intitulé :

Générateur d'examen de type quizz

Présenté par :

AOUAD Abdelkarim & EL OUARDI Abderrahim

Pr. Yahyaouy Ali Professeur (Encadrant)

Année Universitaire 2025–2026

Remerciements

Nous tenons à adresser nos sincères remerciements à notre encadrant, **M. Yahyaouy Ali** , pour son suivi constant durant la réalisation de ce projet. Par ses orientations pertinentes, sa patience et sa grande disponibilité, il nous a motivés à donner le meilleur de nous-mêmes.

Son accompagnement nous a permis d'avancer efficacement dans le développement de notre application, de découvrir de nouvelles approches de travail et de consolider nos compétences techniques.

Nous lui sommes profondément reconnaissants pour le temps, l'attention et l'énergie qu'il nous a consacrés.

Table des matières

Table des matières iii

Liste des figures 1

I Introduction 2

I.1 Contexte 2

I.2 Problématique 2

I.3 Objectifs du projet 3

II Analyse des besoins 4

II.1 Besoins fonctionnels 4

II.2 Besoins non fonctionnels 5

II.3 Acteurs du système 6

II.4 Contraintes techniques 6

III Conception 7

III.1 Diagramme de cas d'utilisation 7

III.2 Diagramme de classes 7

III.3 Diagrammes de séquence 9

III.4 Modèle de données 9

IV Réalisation / Implémentation 11

IV.1 Architecture globale de l'application 11

IV.1.1 Le modèle MVC (Modèle-Vue-Contrôleur) 11

IV.1.2 Architecture technique N-Tiers 11

IV.1.3 Flux de données et Persistance 12

IV.2 Environnement technique et technologies utilisées 12

IV.2.1 Technologies Serveur (Back-end) 12

IV.2.2 Technologies Front-end 12

IV.2.3 Environnement de déploiement et outils 13

IV.3 Description des modules de l'application 13

IV.3.1 Module de Gestion des Examens et Questions 13

IV.3.2 Module de Persistance (DAO) 13

IV.3.3 Module Métier et Logique de Contrôle 14

A Annexes **15**
A.1 Captures d'écran de l'application 15
A.2 Extrait du script SQL 18
A.3 Format des fichiers d'import 18

Webographie **19**

Table des figures

III.1 Diagramme de cas d'utilisation 7
III.2 Diagramme de classes 8
III.3 Diagramme de séquence – Scénario 1 9
III.4 Diagramme de séquence – Scénario 2 9

A.1 Page d'authentification 15
A.2 Page d'accueil de l'administrateur 16
A.3 Page de création d'un examen 16
A.4 Page de passage d'un examen 17
A.5 Interface d'envoi d'email 17

Chapitre I

Introduction

I.1 Contexte

L'évaluation des étudiants dans les établissements universitaires repose souvent sur des méthodes manuelles qui demandent beaucoup de temps (préparation des sujets, convocation, correction, publication des résultats). Dans ce cadre, ce projet vise à développer une application web de génération et de passation d'examens en ligne, basée sur les technologies JSP, Servlet, JavaBean et JPA.

L'application permet de gérer différents types de questions (QCM à réponse unique ou multiple, textes à trous, courte réponse), de constituer des examens, d'inscrire les étudiants via import Excel, puis d'assurer la passation et l'enregistrement automatique des résultats.

I.2 Problématique

La problématique principale est la suivante :

> *Comment concevoir un système d'examen en ligne fiable, flexible et simple d'utilisation, permettant à la fois l'administration complète des examens et une expérience fluide pour les étudiants ?*

Cette problématique inclut plusieurs défis :

- centraliser la gestion des questions et des examens ;
- automatiser l'import des étudiants et l'envoi des accès ;
- garantir une passation sécurisée (authentification, minuterie, ordre aléatoire des questions) ;
- corriger/évaluer rapidement et enregistrer les résultats en base de données.

I.3 Objectifs du projet

Les objectifs de ce projet sont :

- développer une interface d'authentification distinguant administrateur et utilisateur ;
- offrir à l'administrateur la création d'examens avec deux modes d'alimentation des questions :
  - saisie manuelle via interface web ;
  - import via fichier texte respectant une syntaxe définie.
- permettre l'import de la liste des étudiants à partir d'un fichier Excel (nom, prénom, email) ;
- configurer les paramètres d'examen (durée, nombre de questions, liste des candidats) ;
- lancer l'examen et diffuser les liens d'accès aux étudiants ;
- permettre à l'étudiant de passer l'examen en ligne avec affichage séquentiel et aléatoire des questions, ainsi qu'un chronométrage ;
- calculer la note / le nombre de bonnes réponses et sauvegarder les résultats dans la base de données.

Chapitre II

Analyse des besoins

II.1 Besoins fonctionnels

Le système doit couvrir les fonctionnalités suivantes :

- **Authentification et gestion des rôles**
  - Permettre la connexion sécurisée des utilisateurs.
  - Distinguer les profils *administrateur* et *étudiant*.
- **Gestion des questions**
  - Créer, modifier, supprimer des questions.
  - Gérer plusieurs types de questions :
    - QCM (réponse unique / réponses multiples),
    - textes à trous,
    - courte réponse.
- **Création et configuration d'examen**
  - Créer un nouvel examen (titre/thème).
  - Définir la durée de l'examen.
  - Définir le nombre de questions à poser.
  - Associer une liste d'étudiants à l'examen.
- **Alimentation des examens**
  - Ajouter les questions via une interface web.
  - Importer les questions via un fichier texte formaté :
    - QCM : `Question|Choix1, Choix2, ...|NumBonneRéponse`
    - Texte à trous : mots masqués par `.....`
    - Courte réponse : `Question|Rep1,,Rep2,,...`
- **Gestion des étudiants**
  - Ajouter les étudiants manuellement ou via import Excel.
  - Importer les champs : nom, prénom, email.
  - Valider les données importées (format email, doublons, champs obligatoires).
- **Lancement de l'examen**

  - Lancer l'examen pour un groupe d'étudiants ciblé.
  - Générer et envoyer des liens d'accès aux étudiants concernés.
- **Passation de l'examen**
  - Autoriser l'accès par authentification ou lien d'invitation.
  - Afficher les questions une par une.
  - Présenter les questions dans un ordre aléatoire.
  - Afficher un chronomètre pour le temps autorisé.
- **Évaluation et résultats**
  - Calculer automatiquement la note / le nombre de bonnes réponses.
  - Afficher le résultat final à l'étudiant.
  - Enregistrer les résultats dans la base de données.

II.2 Besoins non fonctionnels

Le système doit respecter les exigences de qualité suivantes :

- **Performance**
  - Temps de réponse faible pour les opérations courantes (connexion, affichage des questions, soumission).
  - Capacité à gérer plusieurs sessions d'examen simultanées.
- **Sécurité**
  - Protection de l'authentification (sessions sécurisées, contrôle d'accès par rôle).
  - Protection des données personnelles des étudiants.
  - Prévention des accès non autorisés aux examens.
- **Fiabilité**
  - Sauvegarde fiable des réponses et des résultats.
  - Gestion des erreurs d'import et des interruptions pendant l'examen.
- **Ergonomie**
  - Interfaces simples et intuitives pour l'administrateur et l'étudiant.
  - Navigation claire durant la passation.
- **Maintenabilité**
  - Architecture claire en couches (présentation, contrôle, métier, persistance).
  - Code modulaire facilitant l'évolution (ajout de types de questions, nouvelles règles).
- **Portabilité**
  - Déploiement sur un serveur Java web standard compatible JSP/Servlet.

II.3 Acteurs du système

Les acteurs identifiés sont :

- **Administrateur**
  - Gère les comptes et l'authentification.
  - Crée et configure les examens.
  - Gère la banque de questions.
  - Importe les étudiants (Excel) et les questions (texte).
  - Lance les examens et suit les résultats.
- **Étudiant (Utilisateur)**
  - S'authentifie ou accède via un lien d'invitation.
  - Passe l'examen dans le temps imparti.
  - Consulte sa note finale.
- **Service de messagerie (acteur externe)**
  - Envoie les liens d'accès aux étudiants.

II.4 Contraintes techniques

Le développement doit respecter les contraintes suivantes :

- **Technologies imposées**
  - JSP, Servlet, JavaBean, JPA.
- **Architecture**
  - Architecture web Java EE en couches.
  - Couche DAO/Service pour l'accès et la logique métier.
- **Base de données**
  - Persistance relationnelle avec JPA.
  - Fourniture d'un export de la base (`.sql`) dans les livrables.
- **Interopérabilité des fichiers**
  - Support de l'import Excel pour les étudiants.
  - Support d'un format texte structuré pour l'import des questions.
- **Déploiement**
  - Déploiement sur un conteneur de servlets compatible (ex. Tomcat).

Chapitre III

Conception

III.1 Diagramme de cas d'utilisation

Ce diagramme présente les interactions principales entre les acteurs (administrateur et étudiant) et les fonctionnalités du système.

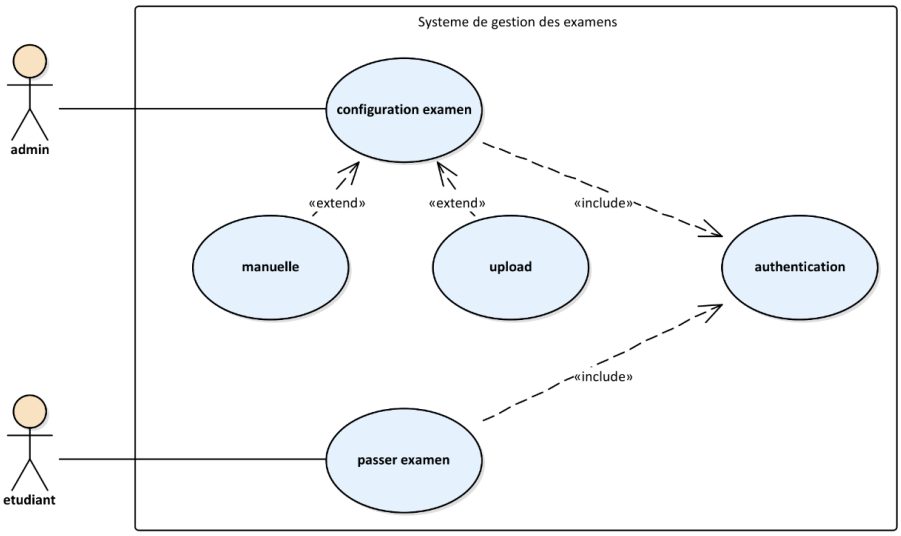

FIGURE III.1 – Diagramme de cas d'utilisation

III.2 Diagramme de classes

Le diagramme de classes décrit la structure statique de l'application : entités métier, attributs essentiels et relations entre classes.

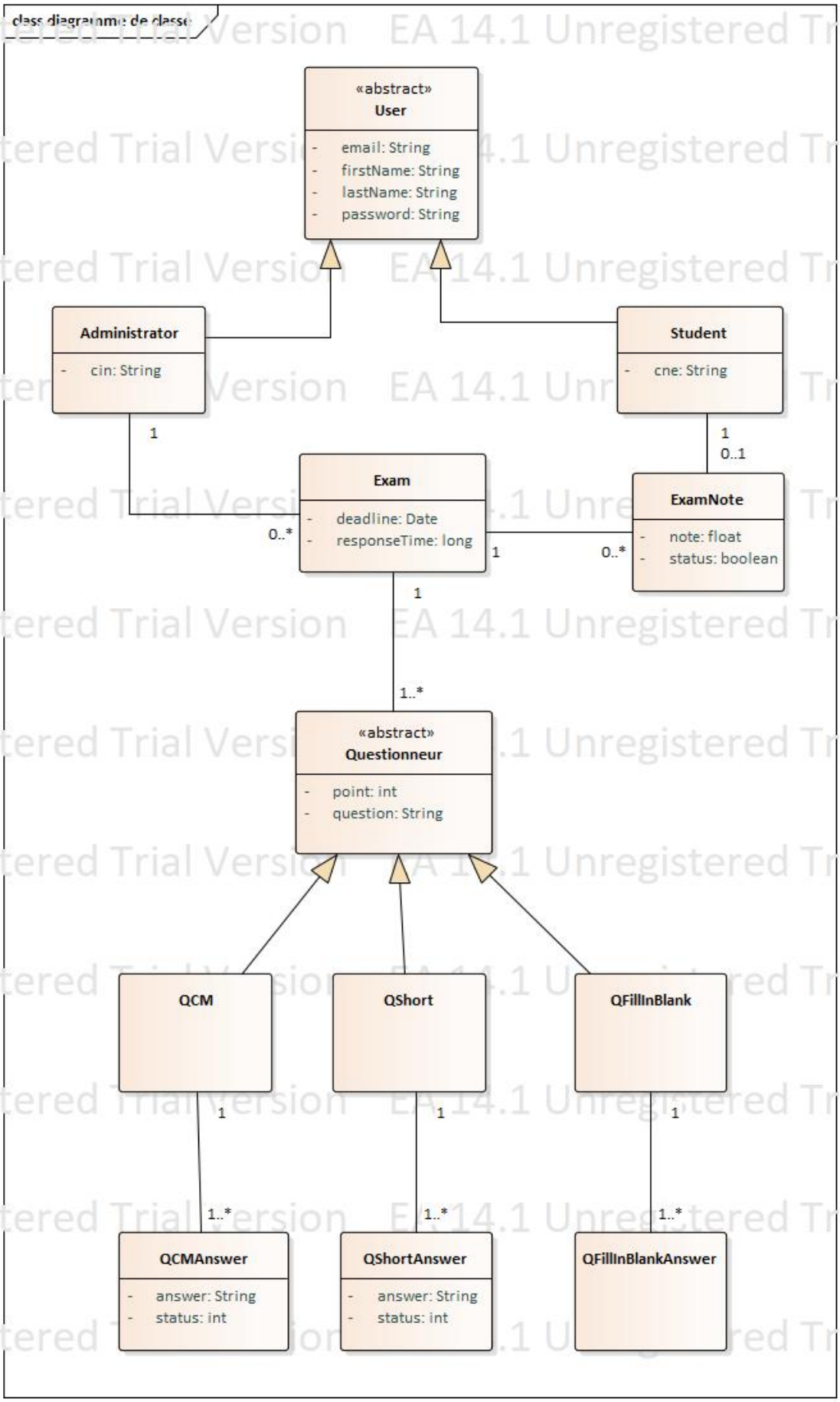

FIGURE III.2 – Diagramme de classes

III.3 Diagrammes de séquence

Les diagrammes de séquence détaillent l'ordre des échanges entre l'interface, les contrôleurs, la couche service et la couche de persistance pour les scénarios clés.

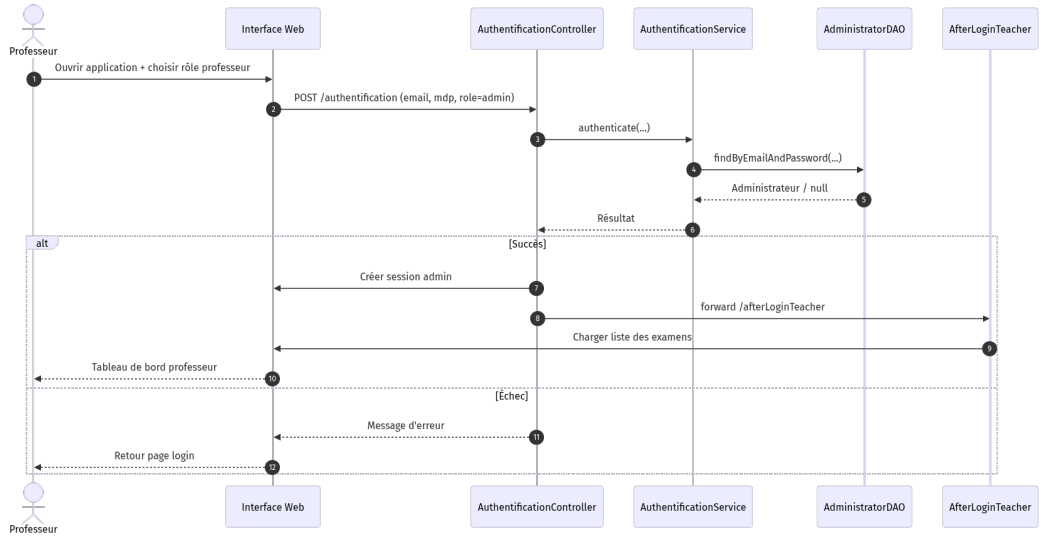

FIGURE III.3 – Diagramme de séquence – Scénario 1

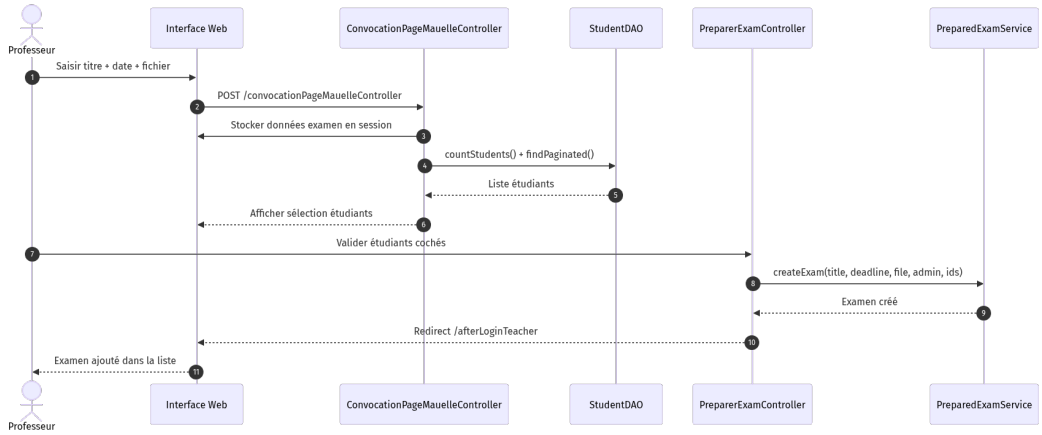

FIGURE III.4 – Diagramme de séquence – Scénario 2

III.4 Modèle de données

Le modèle de données est construit à partir des entités principales du domaine : utilisateurs, examens, questions, propositions, réponses et résultats.

La persistance est assurée via JPA, avec des relations entre entités permettant de représenter :

— l'association entre un examen et sa banque de questions ;

— l'affectation des étudiants aux examens ;

— l'enregistrement des réponses fournies ;

— le calcul et la sauvegarde des résultats finaux.

Chapitre IV

Réalisation / Implémentation

IV.1 Architecture globale de l'application

L'application *ExamsManagement* repose sur une architecture logicielle robuste et évolutive, structurée en couches (Architecture N-Tiers). Cette approche permet de séparer les responsabilités, facilitant ainsi la maintenance et les tests unitaires.

IV.1.1 Le modèle MVC (Modèle-Vue-Contrôleur)

Nous avons adopté le pattern de conception **MVC** pour structurer les interactions utilisateur :

— **Modèle (Model) :** Représente les données et la logique métier. Il est implémenté via des entités *Plain Old Java Objects* (POJO) annotées avec JPA (ex : `Exam`, `Question`).

— **Vue (View) :** Correspond à l'interface utilisateur. Elle est réalisée à l'aide de pages **JSP** (JavaServer Pages) et de technologies Front-end (HTML5, CSS3, JavaScript).

— **Contrôleur (Controller) :** Assure la liaison entre la vue et le modèle. Il est implémenté par des **Servlets** qui interceptent les requêtes HTTP, invoquent la logique métier et redirigent vers la vue appropriée.

IV.1.2 Architecture technique N-Tiers

L'organisation technique de l'application se décline en quatre couches distinctes, déployées sur le serveur d'applications **WildFly** :

1. **Couche Présentation :** Gérée par le navigateur client, elle traite les interactions dynamiques (notamment la génération de formulaires via JavaScript).
2. **Couche Web (Web Tier) :** Contient les Servlets et les filtres de sécurité. Elle gère le cycle de vie des sessions utilisateur.
3. **Couche Métier (Business Tier) :** Utilise les services et éventuellement les EJB pour traiter les règles de gestion complexes, comme le calcul des scores ou la génération aléatoire d'examens.

4. **Couche d'Accès aux Données (DAO Tier) :** Utilise l'API **JPA** avec l'implémentation **Hibernate** pour assurer la persistance des objets dans la base de données relationnelle.

IV.1.3 Flux de données et Persistance

Le mécanisme de persistance est centralisé via le `EntityManager`. Les relations complexes, telles que la liaison `OneToMany` entre un examen et ses questions, sont gérées par des opérations en cascade (*Cascading*), garantissant l'intégrité référentielle lors des transactions.

IV.2 Environnement technique et technologies utilisées

La réalisation de l'application *ExamsManagement* s'appuie sur un écosystème technologique moderne basé sur la plateforme **Jakarta EE** (précédemment Java EE), offrant ainsi scalabilité et robustesse.

IV.2.1 Technologies Serveur (Back-end)

Le cœur de l'application repose sur les spécifications standards de Java pour le Web :

— **Jakarta Servlet & JSP :** Utilisés pour la gestion du cycle de vie des requêtes HTTP et la génération dynamique des vues. Les JSP permettent une séparation claire entre la structure HTML et la logique d'affichage via la JSTL.

— **Jakarta Persistence API (JPA) / Hibernate :** JPA définit le standard de gestion des données relationnelles en Java. Nous avons utilisé **Hibernate** comme fournisseur de persistance (*ORM - Object Relational Mapping*) pour transformer nos classes Java en tables SQL de manière transparente.

— **EJB (Enterprise JavaBeans) :** Utilisés pour encapsuler la logique métier et gérer les transactions automatiques (notamment pour les opérations de sauvegarde complexes).

IV.2.2 Technologies Front-end

Pour garantir une expérience utilisateur fluide, notamment lors de la création manuelle d'examens :

— **HTML5 & CSS3 :** Pour une structuration sémantique et une interface utilisateur moderne et responsive.

— **JavaScript :** Utilisé pour la manipulation dynamique du DOM (Document Object Model), permettant d'ajouter ou de supprimer des questions en temps réel sans recharger la page.

IV.2.3 Environnement de déploiement et outils

Le tableau suivant récapitule les outils de développement et serveurs utilisés :

Composant	**Technologie choisie**
Langage de programmation	Java 17+
Serveur d'applications	WildFly 39 (JBoss)
Système de gestion de base de données	Oracle
Environnement de développement (IDE)	IntelliJ IDEA Ultimate
Gestionnaire de version	Git

Table IV.1 – Récapitulatif de l'environnement technique

IV.3 Description des modules de l'application

L'application *ExamsManagement* est organisée en plusieurs modules fonctionnels interdépendants. Cette modularité assure une séparation nette entre la capture des données, leur traitement et leur stockage définitif.

IV.3.1 Module de Gestion des Examens et Questions

Ce module est le cœur de l'application. Il permet la création et la configuration des examens.

— **Interface Dynamique :** Grâce à JavaScript, ce sous-module permet de générer dynamiquement des formulaires de saisie (QCM, Questions à choix unique) en fonction du nombre de questions souhaité par l'utilisateur.

— **Collecte de données :** Les servlets interceptent les paramètres HTTP indexés (ex : `QCMquestion_i`) pour reconstruire les objets complexes côté serveur.

IV.3.2 Module de Persistance (DAO)

Le module de persistance assure l'interface entre les objets Java et la base de données via JPA.

— **Mapping Objet-Relationnel (ORM) :** Ce module définit les entités (`Exam`, `Question`, `Answer`) et leurs relations (`OneToMany`, `OneToOne`).

— **Gestion des Cascades :** L'implémentation utilise le mode *CascadeType.ALL*, permettant de sauvegarder un examen et l'intégralité de ses questions/réponses associées en une seule transaction atomique.

IV.3.3 Module Métier et Logique de Contrôle

Ce module, situé entre la vue et le DAO, orchestre le flux de l'application :

— **Contrôleurs (Servlets) :** Ils valident les données saisies, instancient les entités et font appel aux services de sauvegarde.

— **Gestion des Sessions :** Assure le suivi de l'utilisateur connecté et maintient l'état de la préparation de l'examen en cours.

Annexe A

Annexes

A.1 Captures d'écran de l'application

Cette annexe regroupe des captures d'écran illustrant les principales interfaces du système.

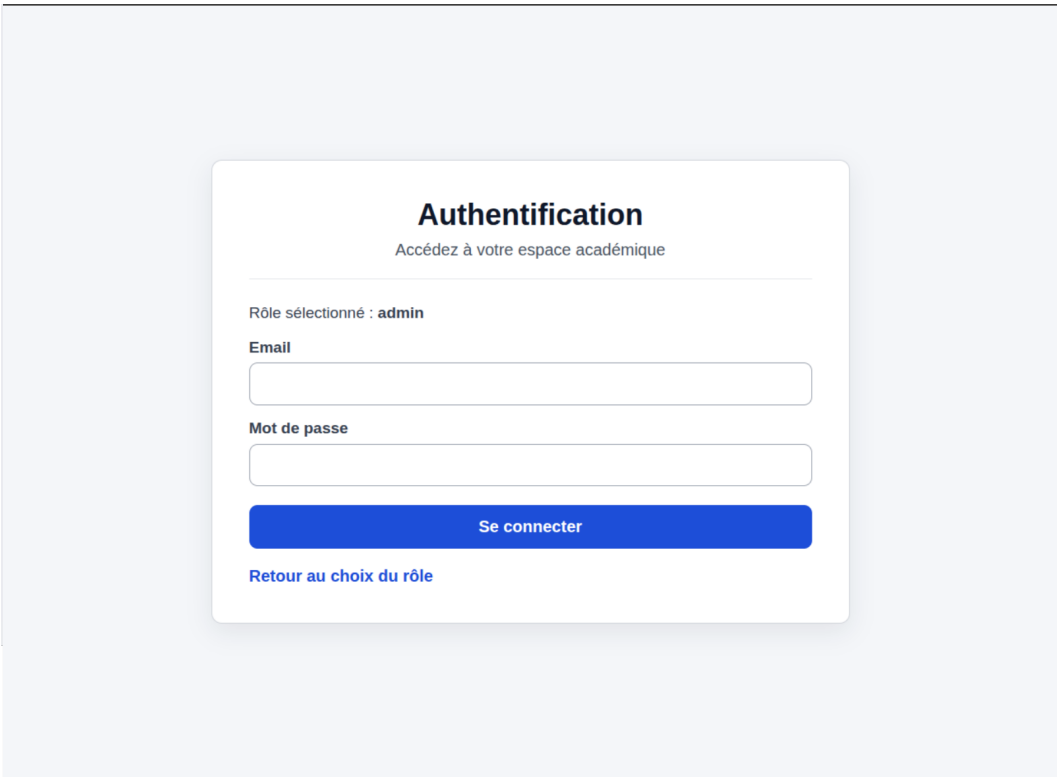

FIGURE A.1 – Page d'authentification

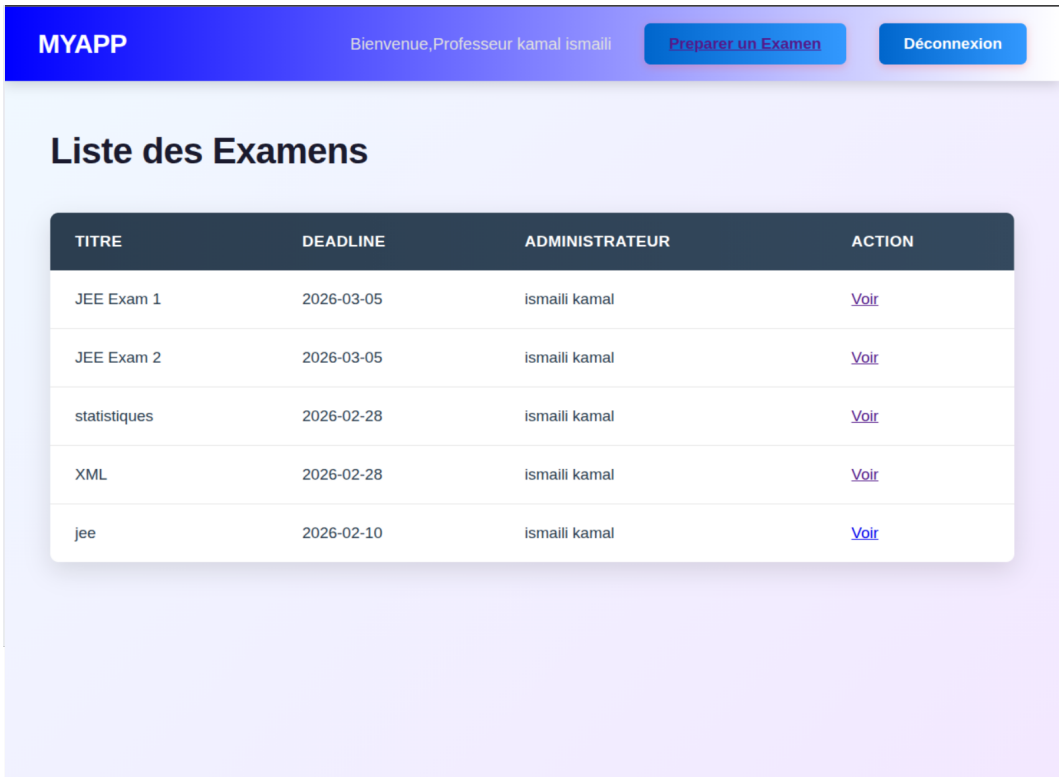

FIGURE A.2 – Page d'accueil de l'administrateur

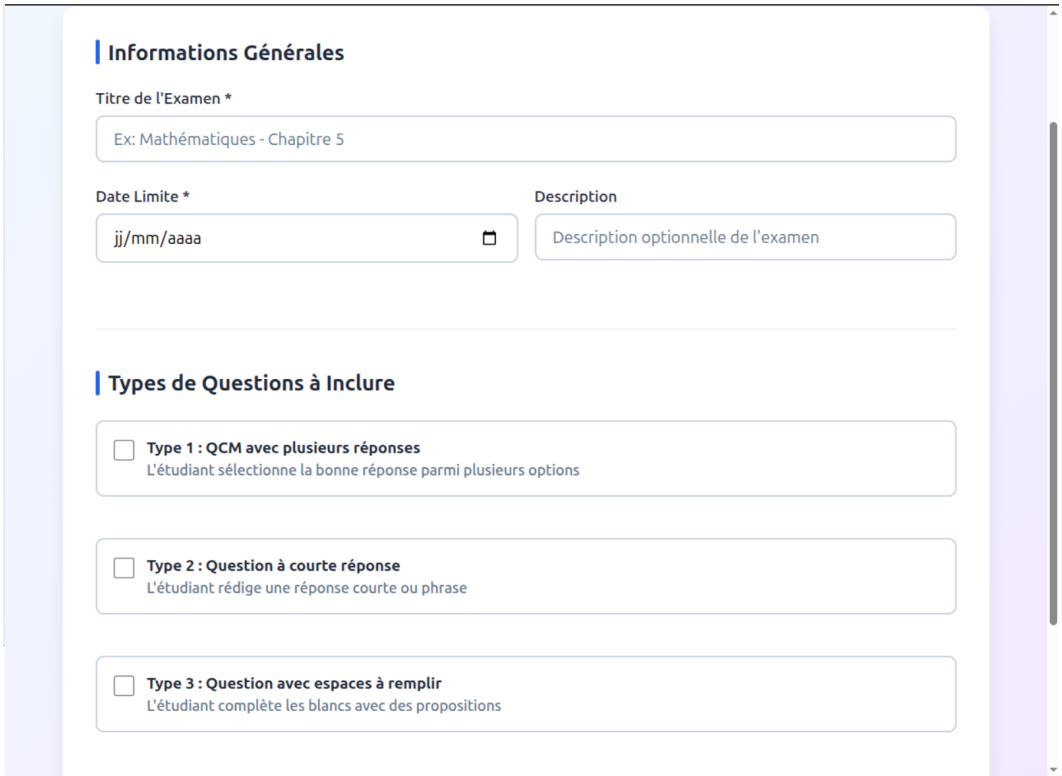

FIGURE A.3 – Page de création d'un examen

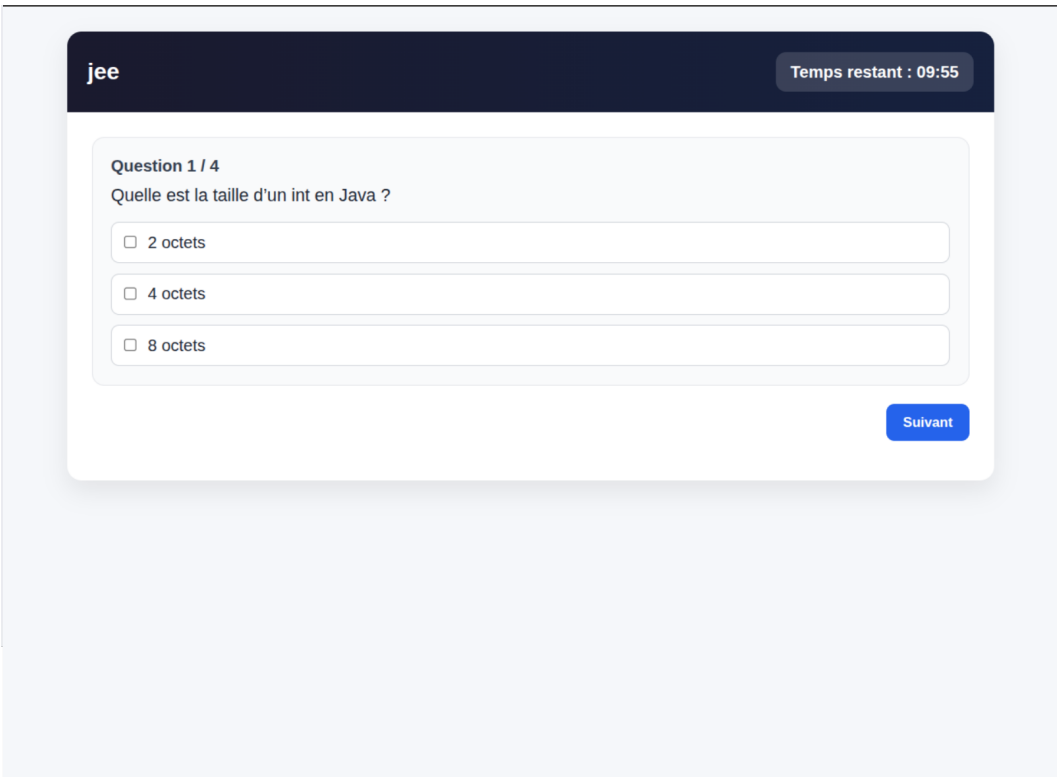

FIGURE A.4 – Page de passage d'un examen

FIGURE A.5 – Interface d'envoi d'email

A.2 Extrait du script SQL

Cette section contient un extrait du script de création de la base de données utilisée dans le projet.

```
CREATE CACHED TABLE "PUBLIC"."APP_USER"(
    "ID" BIGINT GENERATED BY DEFAULT AS IDENTITY(START WITH 1 RESTART WITH 2) NOT NULL
    "EMAIL" CHARACTER VARYING(255),
    "FIRSTNAME" CHARACTER VARYING(255),
    "LASTNAME" CHARACTER VARYING(255),
    "PASSWORD" CHARACTER VARYING(255)
);

CREATE CACHED TABLE "PUBLIC"."QANSWER"(
    "QANSWERID" BIGINT GENERATED BY DEFAULT AS IDENTITY(START WITH 1) NOT NULL,
    "ANSWER" CHARACTER VARYING(255),
    "STATUS" INTEGER,
    "QCMID" BIGINT
);
```

A.3 Format des fichiers d'import

Cette section présente des exemples de formats attendus pour l'importation des données.

Importation des questions

```
Question QCM|Choix1,Choix2,Choix3|2
Question courte|Reponse1,,Reponse2
```

Importation des étudiants

Fichier Excel contenant au minimum les colonnes suivantes : Nom, Prénom, Email.

Webographie

Dans le cadre de la réalisation de ce projet de gestion des examens (Exams Management), plusieurs ressources numériques et documentations officielles ont été consultées pour la mise en œuvre des technologies Jakarta EE, de la persistance JPA et de la gestion des dépendances.

Ressources Techniques et Documentations

— **Jakarta EE Official Documentation**
Consulée pour la mise en œuvre des spécifications Servlets, JSP et Enterprise Java Beans (EJB) au sein du serveur d'applications WildFly.
URL : https://jakarta.ee/

— **Maven Repository (MvnRepository)**
Utilisé pour l'identification et l'intégration des versions stables des bibliothèques (Hibernate Core, Jakarta Persistence API, Connecteur H2/MySQL) via le fichier de configuration `pom.xml`.
URL : https://mvnrepository.com/

— **Hibernate ORM Documentation**
Source principale pour la résolution des problématiques de mapping objet-relationnel, notamment la gestion des associations `@OneToMany`, `@OneToOne` et la gestion du chargement paresseux (*Lazy Loading*).
URL : https://hibernate.org/orm/documentation/

Gestion de Version et Code Source

— **Dépôt GitHub du Projet - Exams Management App**
Hébergement du code source complet, incluant la structure du projet JEE, les entités JPA, les contrôleurs et les vues JSP. Ce dépôt a également servi à la gestion des versions et à la synchronisation du travail via les commandes Git.
URL : https://github.com/abderrahim-elouardi/Exams-Management-App-JEE-JPA

Toutes les ressources mentionnées ont été consultées durant la période du projet (Février 2026).